Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

«Создание таблиц базы данных PostgreSQL. Заполнение таблиц рабочими данными»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Чан Тхи Лиен

Факультет прикладной информатики

Группа K3240

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2023

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

1. Цель работы:

- овладеть практическими навыками установки СУБД PostgreSQL и создания базы данных в pgadmin 4.
- овладеть практическими навыками создания таблиц базы данных PostgreSQL 1X, заполнения их рабочими данными, резервного копирования и восстановления БД. **.**

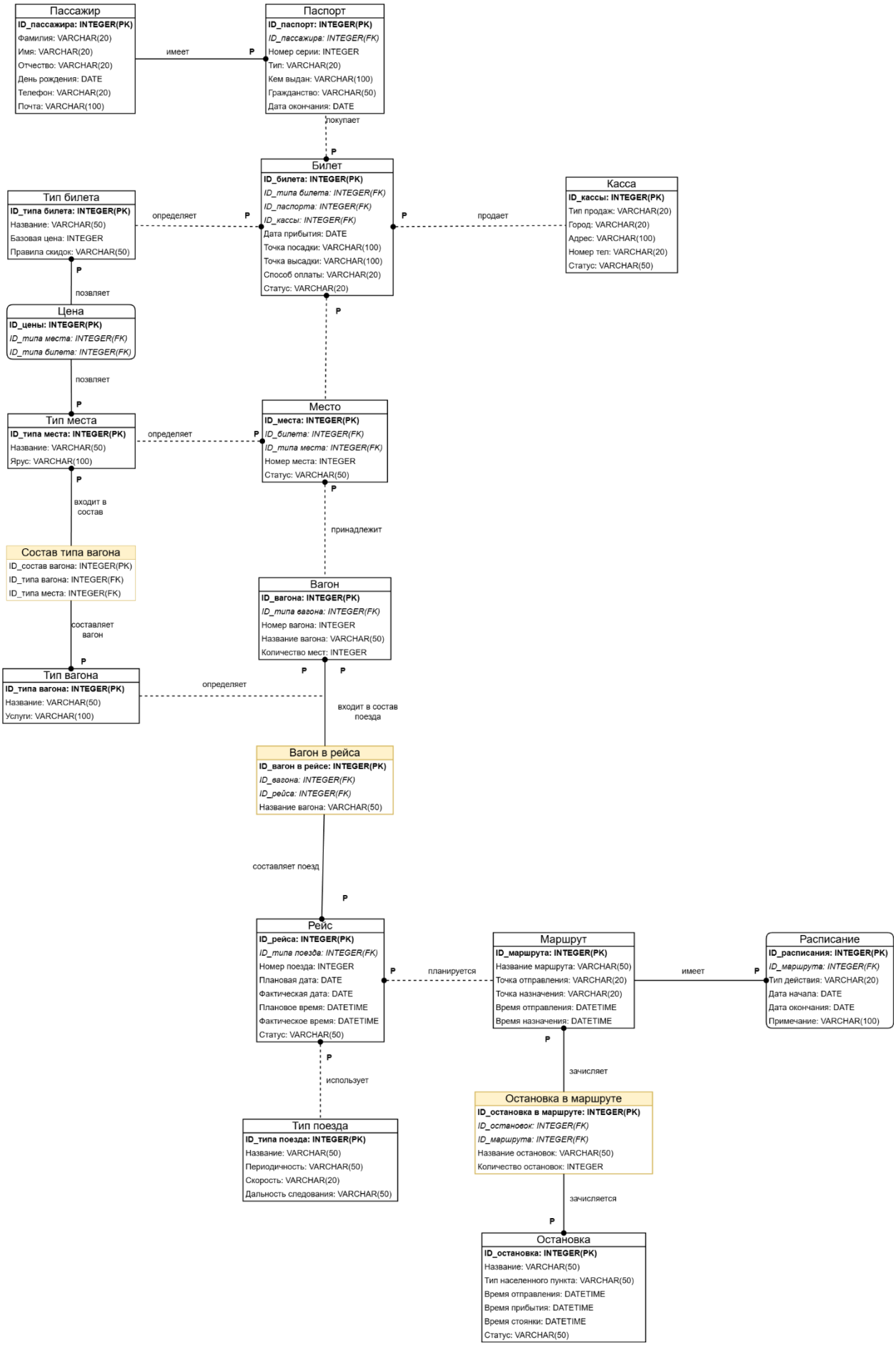
2. Программное обеспечение: СУБД PostgreSQL 1X, pgAdmin 4.

3. Практическое задание

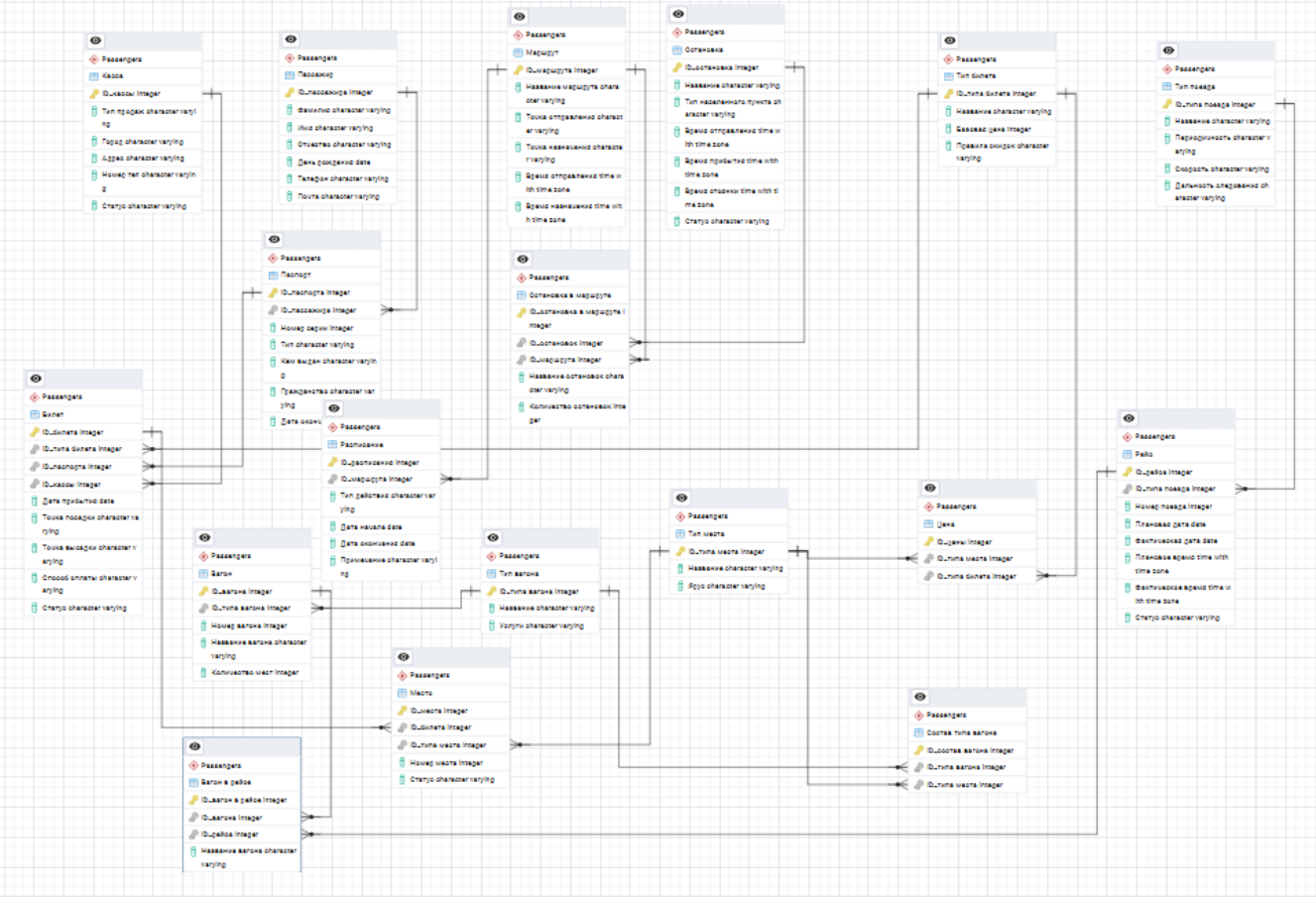
1. Создать базу данных с использованием pgAdmin 4 (согласно индивидуальному заданию).
2. Создать схему в составе базы данных.
3. Создать таблицы базы данных.
4. Установить ограничения на данные: *Primary Key, Unique, Check, Foreign Key*.
5. Заполнить таблицы БД рабочими данными.
6. Создать резервную копию БД.
7. Восстановить БД.

4. Наименование БД «Пассажир» - Вариант 6

5. Схема инфологической модели БД (IDEF1X)


Пассажир
ID_пассажира: INTEGER(PK)
Фамилия: VARCHAR(20)
Имя: VARCHAR(20)
Отчество: VARCHAR(20)
День рождения: DATE
Телефон: VARCHAR(20)
Почта: VARCHAR(100)
имеет
P
Паспорт
ID_паспорт: INTEGER(PK)
ID_пассажира: INTEGER(FK)
Номер серии: INTEGER
Тип: VARCHAR(20)
Кем выдан: VARCHAR(100)
Гражданство: VARCHAR(50)
Дата окончания: DATE
покупает
P
Билет
ID_билета: INTEGER(PK)
ID_типа билета: INTEGER(FK)
ID_паспорта: INTEGER(FK)
ID_кассы: INTEGER(FK)
Дата прибытия: DATE
Точка посадки: VARCHAR(100)
Точка высадки: VARCHAR(100)
Способ оплаты: VARCHAR(20)
Статус: VARCHAR(20)
Тип билета
ID_типа билета: INTEGER(PK)
Название: VARCHAR(50)
Базовая цена: INTEGER
Правила скидок: VARCHAR(50)
определяет
P
P
продает
Касса
ID_кассы: INTEGER(PK)
Тип продаж: VARCHAR(20)
Город: VARCHAR(20)
Адрес: VARCHAR(100)
Номер тел: VARCHAR(20)
Статус: VARCHAR(50)
P
позвляет
Цена
ID_цены: INTEGER(PK)
ID_типа места: INTEGER(FK)
ID_типа билета: INTEGER(FK)
позвляет
P
P
Тип места
ID_типа места: INTEGER(PK)
Название: VARCHAR(50)
Ярус: VARCHAR(100)
определяет
P
Место
ID_места: INTEGER(PK)
ID_билета: INTEGER(FK)
ID_типа места: INTEGER(FK)
Номер места: INTEGER
Статус: VARCHAR(50)
P
входит в состав
принадлежит
Состав типа вагона
ID_состав вагона: INTEGER(PK)
ID_типа вагона: INTEGER(FK)
ID_типа места: INTEGER(FK)
Вагон
ID_вагона: INTEGER(PK)
ID_типа вагона: INTEGER(FK)
Номер вагона: INTEGER
Название вагона: VARCHAR(50)
Количество мест: INTEGER
составляет вагон
P
Тип вагона
ID_типа вагона: INTEGER(PK)
Название: VARCHAR(50)
Услуги: VARCHAR(100)
определяет
P
P
входит в состав поезда
Вагон в рейса
ID_вагон в рейсе: INTEGER(PK)
ID_вагона: INTEGER(FK)
ID_рейса: INTEGER(FK)
Название вагона: VARCHAR(50)
составляет поезд
P
Рейс
ID_рейса: INTEGER(PK)
ID_типа поезда: INTEGER(FK)
Номер поезда: INTEGER
Плановая дата: DATE
Фактическая дата: DATE
Плановое время: DATETIME
Фактическое время: DATETIME
Статус: VARCHAR(50)
P
планируется
Маршрут
ID_маршрута: INTEGER(PK)
Название маршрута: VARCHAR(50)
Точка отправления: VARCHAR(20)
Точка назначения: VARCHAR(20)
Время отправления: DATETIME
Время назначения: DATETIME
имеет
P
Расписание
ID_расписания: INTEGER(PK)
ID_маршрута: INTEGER(FK)
Тип действия: VARCHAR(20)
Дата начала: DATE
Дата окончания: DATE
Примечание: VARCHAR(100)
P
P
использует
зачисляет
Тип поезда
ID_типа поезда: INTEGER(PK)
Название: VARCHAR(50)
Периодичность: VARCHAR(50)
Скорость: VARCHAR(20)
Дальность следования: VARCHAR(50)
Остановка в маршруте
ID_остановка в маршруте: INTEGER(PK)
ID_остановок: INTEGER(FK)
ID_маршрута: INTEGER(FK)
Название остановок: VARCHAR(50)
Количество остановок: INTEGER
зачисляется
P
Остановка
ID_остановка: INTEGER(PK)
Название: VARCHAR(50)
Тип населенного пункта: VARCHAR(50)
Время отправления: DATETIME
Время прибытия: DATETIME
Время стоянки: DATETIME
Статус: VARCHAR(50)

6. Схема логической модели базы данных, сгенерированная в Generate ERD



В классической ИЛМ связи часто строятся на составных (естественных) ключах, где идентификация записи зависит от набора атрибутов родительских сущностей. Однако в вашей реализации БД вы уже применили суррогатные ключи (искусственные идентификаторы типа INTEGER), что соответствует современным стандартам проектирования.

Сущность в ИЛМ	Потенциальный составной ключ в ИЛМ	Суррогатный ключ в реализации
Цена	ID_типа места + ID_типа билета	ID_цены (Primary Key)
Вагон в рейсе	ID_вагона + ID_рейса	ID_вагон в рейсе (Primary Key)
Состав типа вагона	ID_типа места + ID_типа вагона	ID_состав вагона (Primary Key)

Остановка в маршруте	ID_маршрута + ID_остановка	ID_остановка в маршруте (Primary Key)

Использование простых суррогатных ключей:

- Упрощение связей: Внешние ключи (FK) теперь ссылаются только на одну колонку, а не на две или три.
- Производительность: СУБД быстрее обрабатывает соединения (JOIN) по одному целочисленному полю.
- Гибкость: Если бизнес-логика изменится (например, один и тот же тип места в одном типе вагона будет иметь разную стоимость в зависимости от сезона), суррогатный ключ не потребует перестройки всей структуры связей.

7. Dump, содержащий скрипты работы с БД

7.1 Создание базы данных

```
-- 1. КОМАНДЫ СОЗДАНИЯ БАЗЫ ДАННЫХ
-- Перед запуском убедитесь, что вы не подключены к этой БД
DROP DATABASE IF EXISTS "TranLien";
CREATE DATABASE "TranLien"
    WITH OWNER = postgres
    ENCODING = 'UTF8';

-- Переключение контекста на созданную БД (в psql)
\c TranLien
```

7.2 Создание схемы

```sql
-- 2. СОЗДАНИЕ СХЕМЫ
-- Создание логического пространства для таблиц пассажирских перевозок

--
-- TOC entry 6 (class 2615 OID 16389)
-- Name: Passengers; Type: SCHEMA; Schema: -; Owner: postgres
--

CREATE SCHEMA "Passengers";


ALTER SCHEMA "Passengers" OWNER TO postgres;

SET default_tablespace = '';

SET default_table_access_method = heap;
```

7.3 Создание таблиц

```sql
-- 3. СОЗДАНИЕ ТАБЛИЦ

--
-- TOC entry 223 (class 1259 OID 16412)
-- Name: Билет; Type: TABLE; Schema: Passengers; Owner: postgres
--

CREATE TABLE "Passengers"."Билет" (
    "ID_билета" integer NOT NULL,
    "ID_типа билета" integer NOT NULL,
    "ID_паспорта" integer NOT NULL,
    "ID_кассы" integer NOT NULL,
    "Дата прибытия" date,
    "Точка посадки" character varying,
    "Точка высадки" character varying,
    "Способ оплаты" character varying,
    "Статус" character varying,
    CONSTRAINT "check_билет" CHECK (("Дата прибытия" >= '2026-03-31'::date))
);


ALTER TABLE "Passengers"."Билет" OWNER TO postgres;
```

7.4 Ограничения на значения полей

```sql
-- 4. ОГРАНИЧЕНИЯ НА ЗНАЧЕНИЯ ПОЛЕЙ (CHECK)

--
-- TOC entry 4928 (class 2606 OID 16520)
-- Name: Вагон check_вагон; Type: CHECK CONSTRAINT; Schema: Passengers; Owner: postgres
--

ALTER TABLE "Passengers"."Вагон"
    ADD CONSTRAINT "check_вагон" CHECK (("Количество мест" > 0)) NOT VALID;


--
-- TOC entry 4929 (class 2606 OID 16521)
-- Name: Маршрут check_маршрут; Type: CHECK CONSTRAINT; Schema: Passengers; Owner: postgres
--

ALTER TABLE "Passengers"."Маршрут"
    ADD CONSTRAINT "check_маршрут" CHECK (("Время отправления" > "Время назначения")) NOT VALID;
```

7.5 Создание первичных ключей

```sql
-- 5. СОЗДАНИЕ ПЕРВИЧНЫХ КЛЮЧЕЙ

--
-- TOC entry 4940 (class 2606 OID 16420)
-- Name: Билет Билет_pkey; Type: CONSTRAINT; Schema: Passengers; Owner: postgres
--

ALTER TABLE ONLY "Passengers"."Билет"
    ADD CONSTRAINT "Билет_pkey" PRIMARY KEY ("ID_билета");


--
-- TOC entry 4956 (class 2606 OID 16477)
-- Name: Вагон в рейсе Вагон в рейсе_pkey; Type: CONSTRAINT; Schema: Passengers; Owner: postgres
--

ALTER TABLE ONLY "Passengers"."Вагон в рейсе"
    ADD CONSTRAINT "Вагон в рейсе_pkey" PRIMARY KEY ("ID_вагон в рейсе");
```

7.6 Ограничения внешних ключей

```
-- 6. ОГРАНИЧЕНИЯ ВНЕШНИХ КЛЮЧЕЙ

--
-- TOC entry 4970 (class 2606 OID 16600)
-- Name: Билет FK_ID_паспорта; Type: FK CONSTRAINT; Schema: Passengers; Owner: postgres
--

ALTER TABLE ONLY "Passengers"."Билет"
    ADD CONSTRAINT "FK_ID_паспорта" FOREIGN KEY ("ID_паспорта") REFERENCES "Passengers"."Паспорт"("ID_паспорта") NOT VALID;


--
-- TOC entry 4979 (class 2606 OID 16580)
-- Name: Вагон FK_Вагон; Type: FK CONSTRAINT; Schema: Passengers; Owner: postgres
--

ALTER TABLE ONLY "Passengers"."Вагон"
    ADD CONSTRAINT "FK_Вагон" FOREIGN KEY ("ID_типа вагона") REFERENCES "Passengers"."Тип вагона"("ID_типа вагона") NOT VALID;
```

- Constraints: Ограничения первичных ключей (PRIMARY KEY) созданы внутри команд ALTER TABLE. Ограничения внешних ключей (FOREIGN KEY) вынесены в отдельный блок для обеспечения ссылочной целостности.

7.7 Запросы на вставку данных

Query Query History

```
1  INSERT INTO "Passengers"."Касса"(
2      "ID_кассы", "Тип продаж", "Город", "Адрес", "Номер тел", "Статус")
3      VALUES (10, 'Офлайн', 'Москва', 'Вокзальная пл., 1', '123-45-67', 'Работает');
```

Data Output Messages Notifications

```
INSERT 0 1

Query returned successfully in 52 msec.
```

Query Query History

```
1 SELECT * FROM "Passengers"."Касса";
```

Data Output Messages Notifications

	ID_кассы [PK] integer	Тип продаж character varying	Город character varying	Адрес character varying	Номер тел character varying	Статус character varying
1	10	Офлайн	Москва	Вокзальная пл., 1	123-45-67	Работает

Полный файл dump, содержащий скрипты работы с БД

https://drive.google.com/file/d/1vPoiVhT4KgyQN0H_fpPDc0HABdaKZX1G/view?usp=sharing

8. Вывод

В ходе выполнения лабораторной работы была успешно осуществлена трансформация инфологической модели (ИЛМ) в физическую структуру базы данных. Основное внимание было уделено переходу от концептуального описания сущностей к их программной реализации в среде pgAdmin 4.

Ключевые результаты работы:

- Оптимизация структуры ключей: В соответствии с принципами физического проектирования, составные ключи сущностей ИЛМ (например, в таблицах «Цена», «Вагон в рейсе», «Остановка в маршруте») были заменены на простые суррогатные ключи типа INTEGER(PK). Это позволило упростить связи между таблицами и повысить надежность ссылочной целостности.
- Обеспечение целостности данных: Реализована многоуровневая система ограничений, включая первичные ключи (PRIMARY KEY), внешние ключи (FOREIGN KEY) для связи таблиц и проверочные ограничения (CHECK). В частности, настроены логические проверки дат (например, для таблиц «Паспорт» и «Билет») и числовых значений (для таблицы «Тип билета»).

- Физическая реализация: Создана полная схема базы данных Passengers, включающая 17 взаимосвязанных таблиц, что полностью покрывает функциональные требования предметной области: от учета паспортных данных пассажиров до формирования сложных маршрутов и расписаний движения поездов.
- Практические навыки: Отработаны навыки работы с инструментом Backup для генерации SQL-дампов и инструментом ERD Tool для визуализации и анализа связей в базе данных.

Таким образом, созданная база данных является надежным фундаментом для дальнейшей разработки информационной системы, обеспечивая корректное хранение данных и высокую скорость выполнения запросов на выборку и модификацию данных в последующих работах.